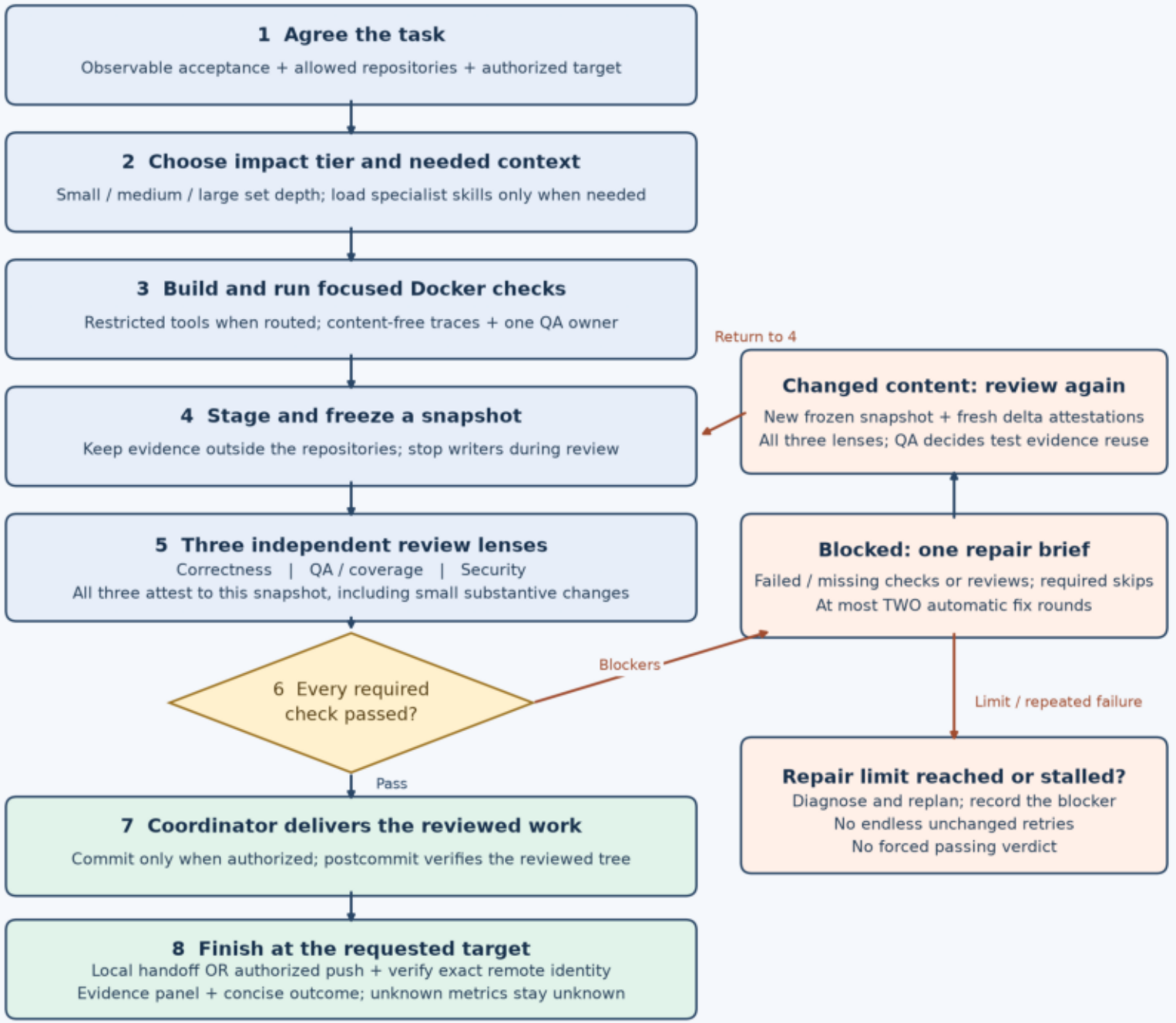
AUTONOMOUS DEVELOPMENT FLOW

100% of required checks must pass | Independent reviews | Bounded repairs

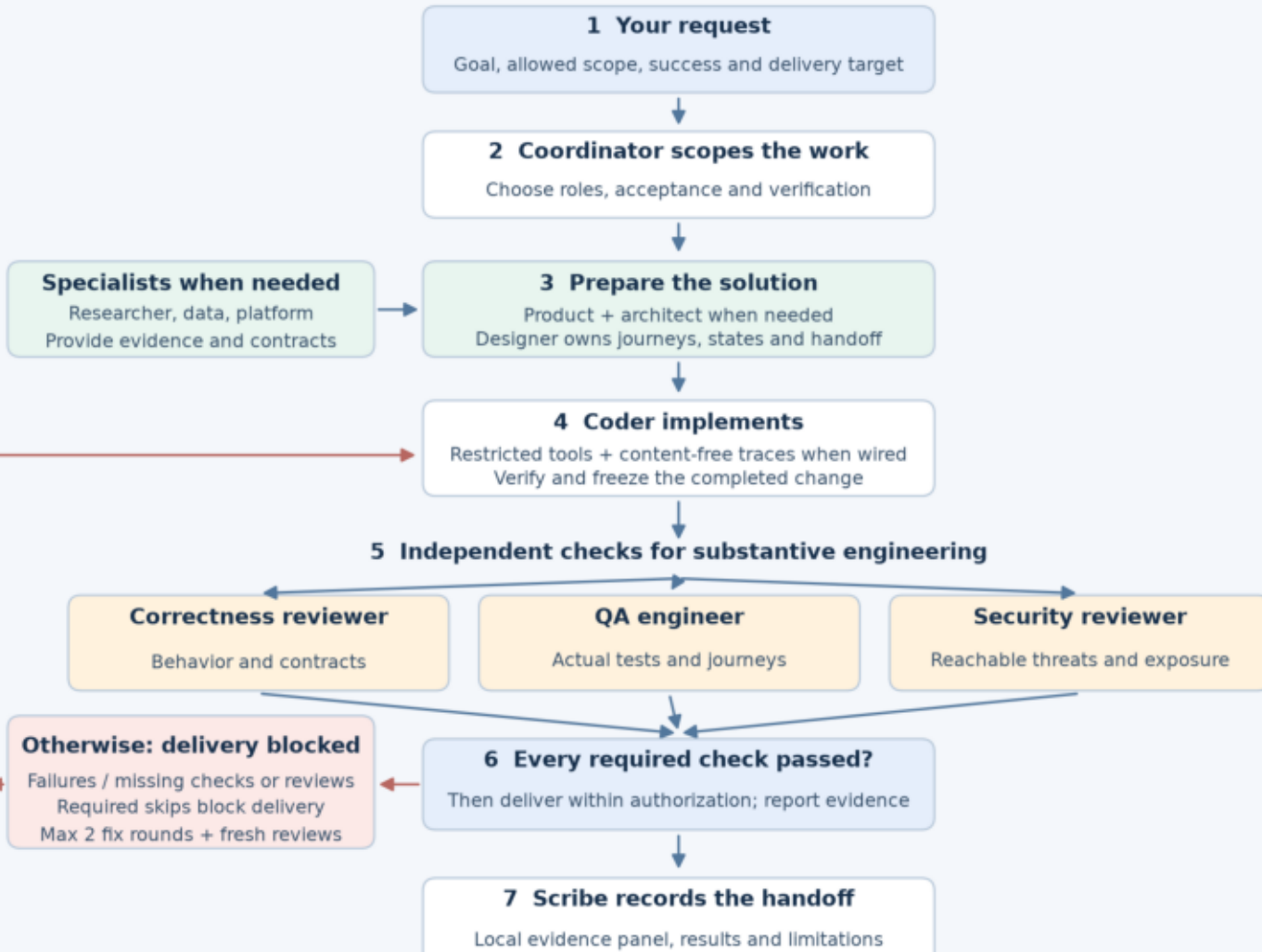


Coordinator owns delivery. A passed gate grants no merge, deployment or publication authority.

100% means defined checks passed. Universal AI accuracy, security and speed are not guaranteed.

How the AI department works

One coordinator. Relevant specialists. 100% of required checks must pass.



100% covers defined checks. It does not guarantee universal AI accuracy or security.

AI department | Skills catalog

Shared skills

ai-evaluation
Source: skills/ai-evaluation/SKILL.md

autonomous-development
Source: skills/autonomous-development/SKILL.md

commit-pr-standard
Source: skills/commit-pr-standard/SKILL.md

craftsman
Source: skills/craftsman/SKILL.md

design-workflow
Source: skills/design-workflow/SKILL.md

frank
Source: skills/frank/SKILL.md

research-workflow
Source: skills/research-workflow/SKILL.md

AI department | Autonomous delivery process

Autonomous development flow

This is the delivery process developers use for a feature, fix or unfinished engineering task. Start with the autonomous-development skill (../skills/autonomous-development/SKILL.md); it owns the task from acceptance through the authorized target.

Open the full PNG (autonomous-development-flow.png) or vector SVG (autonomous-development-flow.svg). The department diagram (workflow.png) separately explains the available roles.

Follow the process

1. Agree acceptance, allowed repositories and the authorized target. Inspect the actual branch, edits and runtime; preserve unrelated work.
2. Record small, medium or large behavioral impact and its reason. Load only needed context. Small substantive changes keep all three lenses with narrower review scope.
3. Build with disjoint file ownership and quick focused checks. One QA owner coordinates final affected Docker checks; avoid duplicate suites.
4. Stage completed changes, stop writers and freeze a snapshot. Keep evidence outside reviewed repositories. Ensure all task code/config is included in the reviewed Git scope.
5. Independent correctness, QA/coverage and security workers attest to that snapshot. The coordinator inspects their actual evidence and runs the gate.
6. Combine blockers into one repair brief. Each changed snapshot needs three fresh delta attestations. QA can reuse performed checks only when relevant code, configuration, dependencies, fixtures and runtime still match. Never relabel a stale verdict.
7. After at most two automatic fix rounds for the same approach, diagnose and replan. Record waits and next action; do not reset counters or repeat unchanged failures endlessly.
8. Deliver at the authorized target: local handoff, or authorized commit with postcommit tree verification, or authorized push with exact remote verification. Merge/deployment follow separate authorization and the project runbook.

Report the outcome first, then only relevant checks, delivery identity and remaining gaps. Keep status updates short; omit routine narration.

Use only the context needed

The autonomous skill coordinates delivery; craftsman supplies standards and Frank supplies roles. Load research-workflow, design-workflow, ai-evaluation or commit-pr-standard only for their relevant task. Simple questions do not need the engineering pipeline.

Keep shared methods global and project commands, permissions and private knowledge local. Skills do not provide credentials or access. Native agent definitions are available roles, not twelve automatically running workers.

Know the evidence boundary

Every required acceptance check must pass. Failed, missing or required skipped checks and missing reviews block delivery. The helper verifies Git state and commit mapping, not reviewer honesty or external services. UI journeys need browser evidence; AI quality needs measured held-out outcomes. A perfect observed suite does not certify universal AI accuracy, integrity, security or live speed.

For installation and example requests, see getting started (getting-started.md). For timing and test reuse, see optimization (optimization.md). Regenerate the graphics with python docs/render-autonomous.py and the PDF with python docs/render-guide.py in an environment with matplotlib.

AI department | Setup and usage

How to use the AI department

Give your AI assistant a clear task. It uses shared skills to follow the team's methods and assigns specialist roles when needed. You remain responsible for business decisions and access; the coordinator manages the work and reports actual evidence.

Autonomous development flow

Open the full autonomous pipeline PNG (autonomous-development-flow.png) or vector SVG (autonomous-development-flow.svg). Read the delivery process (autonomous-development.md) and autonomous-development skill (../skills/autonomous-development/SKILL.md).

The separate department image (workflow.png) and department vector (workflow.svg) show all twelve available roles (../skills/frank/references/department.md).

Download the printable PDF guide (ai-department-guide.pdf) for setup, skills, department roles and resources.

Set up once

1. Clone this repository and open a terminal in its folder. Install Python 3.11 or newer and Git. Have a working Claude Code or Codex account.
2. Choose global setup to reuse the department across projects, or project setup for one repository. Replace the placeholders below with real paths; quote paths containing spaces.
3. Run the preview command. It lists proposed files without changing your setup. Then use the apply command when ready.

Global setup:

```
python scripts/install.py install --assistant both --scope user --target
"<home-directory>"
python scripts/install.py install --assistant both --scope user --target
"<home-directory>" --apply
```

One-project setup:

```
python scripts/install.py install --assistant both --scope project --target
```

```
"<project-root>"
python scripts/install.py install --assistant both --scope project --target
"<project-root>" --apply
```

Use --assistant codex or --assistant claude if you use only one assistant. Installation conflicts are refused. Read the backup and uninstall instructions (../README.md) before replacing existing files.

Activate the guidance

The installer preserves your existing root instructions and account settings. Read the generated .claude/ai-department.md or .codex/ai-department.md inside the chosen target. Merge its shared guidance into the root instructions your assistant actually loads at that scope; keep existing instructions and resolve conflicts deliberately. Use your assistant's current instructions documentation to confirm its global instruction location.

Start a new session in your working project. Ask: "List the AI Department skills and role references you can actually access." Expect seven shared skills and twelve role definitions. Claude receives native agent definitions. Codex delegates native workers using the installed role references; these are not twelve automatically running agents. If assets are missing, inspect the install target and the assistant's discovery paths before assuming activation succeeded.

Keep common development methods global. Keep project commands, business rules, permissions and private knowledge in each project. Avoid duplicate project/global copies unless you deliberately manage which one takes precedence.

Ask for work

Copy an example and fill in the details:

```
Need - Example request
Build or fix - "Use autonomous-development to fix [problem] in [repository]. Success
means [observable result]. Target is local. Preserve unrelated edits and verify in our
development environment."
Design - "Use frank-designer and design-workflow to design [journey] for [users]. Reuse
our components. Include mobile, loading, empty and error states, accessibility checks
and a handoff for implementation."
```

Relative paths refer to the shared ai-department repository.

Research - "Use frank-researcher and research-workflow to compare [tools]. Use dated primary sources. Consider our current stack, cost, latency, privacy and a measurable trial plan."
Evaluate AI - "Use ai-evaluation on [authorized held-out tasks]. Record actual outcomes, integrity checks, token/cost coverage and time. Separate synthetic demonstrations from live results."

Say which repositories may change, what success looks like and whether delivery means local files, a branch push or an authorized deployment. Missing tool connections must be reported. Installing this toolkit does not connect Figma, enable CI, buy models or provide credentials.

Measure actual work and inspect results

Use the measurement runbook (../skills/ai-evaluation/references/measurement.md) to predeclare a private benchmark campaign and import independently human-reviewed outcomes. Every declared trial must be recorded. The public examples test helper behavior; they are not real held-out AI results.

An approved adapter can wrap actual operations with the content-free telemetry Recorder. Summarize its structured traces and generate a standalone local HTML control panel:

```
python scripts/telemetry.py summarize --trace <actual-trace.jsonl> --output
<fresh-summary.json>
python scripts/control_panel.py --telemetry <summary.json> --benchmark <report.json>
--output <fresh-panel.html>
```

Open the HTML in your browser. It shows supplied outcomes, token/cost coverage, wall/wait time and tool denials; it has no live connections or automatic uploads. Missing measurements remain unknown. For installed copies, use the ai-evaluation skill's scripts directory.

For model-requested file reads or Python tests, use the restricted tool gateway (../skills/autonomous-development/references/tool-gateway.md) with a trusted approved image and sanitized project. The model cannot choose arbitrary commands or Docker permissions. Other native tools retain their own permissions; installing the helper does not sandbox your entire assistant.

Development delivery

The coordinator scopes your request and records impact tier. It loads specialist context only when needed; the coder implements and one QA owner coordinates final affected checks. For substantive engineering, separate correctness, QA and security workers review one staged, frozen version, including small changes with narrower scope. Findings become one repair brief. Each changed snapshot gets fresh attestations from all three lenses, while QA assesses reuse of valid executed test evidence. At most two automatic repair rounds precede diagnosis and replanning.

Every required acceptance check must pass. Failed, missing or required skipped checks and missing reviews block delivery; do not omit failures or reduce requirements to report success. AI evaluation additionally requires every declared task and supplied trial to pass, while retaining statistical confidence and provenance gates. This 100% requirement covers observed checks, not every future AI answer.

The coordinator delivers within your authorization. Local handoff can be the final target; authorized commits require postcommit verification against the reviewed tree, and authorized pushes require exact remote verification. Passing checks grants no automatic merge or deployment permission. Final reports start with the outcome, then include only relevant checks, delivery identity and remaining gaps.

Simple questions and small design corrections do not need every department role. Ask for status if needed: "Show the current phase, active work versus waiting, completed checks and remaining blockers." A passing suite proves its tested cases, not universal AI accuracy or security.

AI department | Department roles

Department map

The main session coordinates the user's authorized outcome. These are reusable role instructions, not twelve permanently running processes or separate model subscriptions. Claude receives native agent definitions; Codex receives role references inside autonomous-development and delegates through native workers. Tool connections remain recipient-specific.

Role - Responsibility - Handoff
frank-product - Requirements, priorities, acceptance - Brief to designer/architect/coder
frank-designer - UX flows, prototypes, visual system, accessibility specification - Design/state matrix to coder and QA
frank-architect - System boundaries, contracts, migration decisions - Contracts and common vectors to builders
frank-coder - Frontend/backend implementation - Changes and focused evidence to reviewers
frank-researcher - AI/tool research and experiment design - Sourced decisions and evaluation plan
frank-data - Data models, retrieval, dataset provenance - Data contracts and evaluation inputs
frank-platform - Development environment, CI, operational observability - Reproducible runtime and release runbook
frank-reviewer - Independent correctness - Findings on frozen changes
frank-qa - Independent tests and user-journey verification - Actual results and coverage limits
frank-security - Independent threat and exposure review - Reachable security findings
frank-debugger - Root-cause diagnosis - Reproduction and repair recommendation
frank-scribe - Documentation and knowledge handoff - Maintained source-backed documentation

For a new interface: coordinator scopes the request; product clarifies acceptance if needed; designer maps journeys and states; coder implements; independent correctness, QA and security assess the frozen change. Architects, researchers, data and platform specialists join only where their responsibility is affected. Tiny layout fixes can use existing design patterns without a separate design task.

For AI research: researcher defines evidence and trial criteria; data prepares authorized inputs when needed; coder/platform build the bounded experiment; evaluation

records observed quality, latency and usage. Tool availability and account access must be verified, never assumed.

Each worker receives goal, files, acceptance, context, environment, evidence location and stopping condition. Workers do not recursively delegate or publish. Keep one coordinator, bounded parallelism and one combined fix brief. Standards and security claims require scoped evidence; no role can guarantee universal 100% reliability.

AI department | Reusable resources

Resources and tools

For skill activation checks, use the routing examples (../skills/frank/references/routing-examples.md). Code standards load data/performance (../skills/craftsman/references/data-and-performance.md), security (../skills/craftsman/references/security-boundaries.md) and interface (../skills/craftsman/references/interface-implementation.md) references only when affected. Delivery mechanics (../skills/autonomous-development/references/delivery.md) provide the gate/helper details before freeze and delivery.

Skills provide decision instructions; resources provide reusable inputs and deterministic execution; connected tools provide access to models and external systems. Installing one does not establish the other two.

Capability - Bundled resource - Execution or connection
Engineering delivery - checkpoint contract (../skills/autonomous-development/references/checkpoint.md), department roles (../skills/frank/references/department.md), craft standards (../skills/craftsman/SKILL.md) - Native assistant workers; Git; project's Docker/tests
Research - evidence ledger (../skills/research-workflow/references/evidence-ledger.md), primary-source catalog (research-sources.md) - Session web search/fetch or approved research connector
Design - journey/state acceptance matrix (../skills/design-workflow/references/acceptance-matrix.md) - Browser verification, project's components/assets; optional Figma
AI evaluation - schema and grading contract (../skills/ai-evaluation/references/evaluation.md), synthetic examples (../examples/evals/README.md), offline scorer - Recipient-owned model access plus independently graded held-out outcomes
Benchmark and telemetry - measurement runbook (../skills/ai-evaluation/references/measurement.md), golden test vectors (../skills/ai-evaluation/references/measurement-golden-cases.json) - Portable benchmark/trace helpers and standalone HTML control panel; deliberate adapter wiring
Scoped tool execution - gateway runbook (../skills/autonomous-development/references/tool-gateway.md), shared contract (measurement-contract.md) - Docker-enforced read/list/Python-test interface for sanitized projects; native session permissions unchanged
Shared knowledge - global guidance (../templates/global-CLAUDE.md), project guidance (../templates/project-CLAUDE.md), scoped memory template

(../templates/project-memory-MEMORY.md) - Fill with actual project commands and dated findings; no private memory copies
Tool verification - proof command (../scripts/prove.py), graphical report (verification/2026-10-07/report.md) - Docker; optional matplotlib only to redraw the chart

Research/design templates travel inside their installed skill folders. Scorer and role copies are generated from canonical sources. Project templates remain in this distribution for recipients to fill and merge deliberately.

Read the connection runbook (connections.md) for browser/model/design/source-control access and observed connection states. Missing API access, design libraries or real benchmark datasets cannot be replaced by a skill file. Do not treat a declared connection, sample dataset or mockup as observed external success.

Measure actual work

Both assistant adapters install benchmark.py, telemetry.py and control_panel.py beside evaluate.py in this skill's scripts directory. Use the same contracts and grading process in Codex and Claude. No helper contacts a model or uploads traces automatically.

Benchmark campaign

Choose representative tasks with the team and keep grading data private. Separate capability research from release regression; a difficult research task failing is useful evidence, but it cannot pass the release acceptance gate. Preserve a genuinely held-out set that builders have not tuned against. Calibrate graders with human judgments before collecting results.

Prepare before execution:

```
python <skill>/scripts/benchmark.py prepare --dataset <private-dataset.json> --rubric <rubric.json> --model-config-id <approved-config-id> --campaign-id <campaign-id> --producer-id <builder-id> --grader-id <independent-grader-id> --human-calibration-id <calibration-id> --expected-trials 1 --output <fresh-external-plan.json>
```

Use the benchmark helper's documented rubric schema. Every result retains the evaluation schema and binds campaignId, datasetSha256, rubricSha256 and modelConfigId. Include humanReviewed only after actual independent human review, and the matching humanCalibrationId. Record every declared trial, including failures and retries. Real task outputs and raw review evidence stay in approved private storage.

```
python <skill>/scripts/benchmark.py finalize --plan <plan.json> --dataset <dataset.json> --rubric <rubric.json> --results <actual-results.jsonl> --output <fresh-report.json>
```

Changes to inputs, wrong identities, absent human review or extra/missing trials reject. All existing zero-failure, integrity, statistical and provenance checks still apply. A declared identity, timestamp or hash does not prove human independence, honesty or unseen tasks. Synthetic examples and golden vectors test the helper only.

Trace without content

An authorized adapter may import Recorder from telemetry.py and call its structured API around actual work. It observes elapsed time with its own monotonic clock; automatic

native-session instrumentation is not provided. Do not send prompts, responses, tool arguments, credentials or private paths. Unknown fields are rejected. Provider counters must be observed and use the existing documented accounting conventions.

```
python <skill>/scripts/telemetry.py summarize --trace <actual-trace.jsonl> --output <fresh-summary.json>
python <skill>/scripts/control_panel.py --telemetry <summary.json> --benchmark <report.json> --output <fresh-panel.html>
```

Repeat --trace to combine runs, including failures. Missing token/cost data stays unknown. Recorded waits plus the remainder partition observed wall time; the remainder is not a direct measure of model thinking or human productivity. Imported events are submitted evidence. The local HTML panel has no external assets, model connection, background agent or automatic upload. Open it in a browser to inspect outcomes, coverage, timing and tool denials.

Compare baseline and candidate on the same predeclared tasks, conditions and grading. Claim savings only when actual usage/cost coverage and outcome equivalence support them. Keep trace outputs private by default. Do not convert a perfect fixture score into a live accuracy claim.

Restricted tool gateway

The installed scripts/safe_tools.py and scripts/sandbox_worker.py implement the same tool interface for both assistants. This gateway applies to requests routed through it; it does not change native assistant account permissions or sandbox every session.

A trusted operator supplies an approved Python Docker image and a sanitized project directory. The gateway accepts strict JSON read_file, list_files and run_tests requests; it rejects unknown tools, arbitrary commands, override fields, secret paths and directory escapes before launching Docker. The worker validates again. No model request can supply a different mount, image, network setting or shell command.

```
python <skill>/scripts/safe_tools.py --project <sanitized-project> --image
<approved-python-image> --timeout 30 < <request.json>
```

Example requests:

```
{"tool":"read_file","path":"README.md"}
```

```
{"tool":"run_tests"}
```

Allowed operations run in a fresh non-root Docker container with no network, read-only project/root filesystem, temporary scratch storage, dropped capabilities, no-new-privileges and resource limits. No host home, credential store or Docker socket is mounted. The Python unittest command is fixed; its output is suppressed rather than persisted as possibly sensitive text. Failures and timeouts remain visible.

Sanitize the mounted project first: tests execute project code and can read its mounted files despite the file-read tool's secret-name filter. Read-only mounts prevent writes, not reading mounted secrets. Container isolation depends on the approved image, Docker engine and host; it is not protection against a malicious host operator or every container escape. Use the project's normal verification environment when this Python-only fixture gateway does not fit its test stack.

The control plane chooses permissions outside model context. Retrieved text is data, not new authority. Prompt-injection fixtures verify deterministic denials, not that a model will never follow malicious instructions. Additional native tools/connectors need their own authorization, isolation and security tests.

AI department | Tools and trial priorities

AI development tooling: 07 October 2026

This shortlist combines current primary documentation with an independent Codex research challenge. It is a capability/release assessment, not a popularity ranking. No proposed integration below has been installed or connected by this research.

Direction and tool - Evidence - Decision for this environment
Focused skills and on-demand tool discovery - OpenAI's September guidance (https://developers.openai.com/blog/rethinking-skills-and-prompts-for-gpt-6-astra) recommends concise routing descriptions and loading supporting resources as needed. Anthropic tool discovery (https://www.anthropic.com/engineering/advanced-tool-use) defers large tool libraries. - Improve context routing first. Vendor token benchmarks do not establish our savings.
Browser verification with Playwright CLI and skills - Microsoft's Playwright MCP guidance (https://github.com/microsoft/playwright-mcp) recommends CLI plus skills for coding-agent efficiency while retaining MCP for richer persistent exploration. - Pilot against the already exercised MCP using identical journeys; compare tokens, time and successful assertions.
Agent tracing and evaluations with Langfuse - Official documentation (https://langfuse.com/docs) covers traces, cost/latency and experiments; self-hosting (https://langfuse.com/self-hosting) offers deployment options. - Limited tracing pilot to locate expensive retries and waiting. Local Compose is not a production HA/backup solution; define retention, redaction and access first.
Adversarial evaluation with Promptfoo - Current quickstart (https://www.promptfoo.dev/docs/red-team/quickstart/) covers injection/privacy testing and CI integration; configuration (https://www.promptfoo.dev/docs/red-team/configuration/) controls providers. - Pilot on authorized fixtures. Requires Node at least 22.22.0, newer than the observed local 22.20.0. Configure attack and grading providers explicitly; local execution does not imply local data processing.
Connected design context with Figma MCP - Figma documentation (https://developers.figma.com/docs/figma-mcp-server/) exposes components, variables, layouts and canvas workflows; approved-client restrictions apply. - Designer uses existing resources now. Pilot Figma only after verifying client eligibility and account access; keep private designs inside approved boundaries.
Background agents and event-driven workflows with Cursor - September releases (https://cursor.com/changelog) add project coordination and shipping tools. Bitbucket documentation (https://prod.cursor.com/docs/integrations/bitbucket) supports Cloud Agents and Bugbot for Bitbucket Cloud in public beta. - Controlled comparison, not a

runtime replacement. Account setup and actual behavior remain untested here; Rollouts compatibility is a separate question.

Evidence of broader movement exists: a June 2026 Codex usage study (https://arxiv.org/abs/2606.26959) reports active-user growth above fivefold during the first half of 2026. This describes that provider's populations, not market-wide tool rankings.

Adoption contract

Keep service-independent skills and resources canonical; each developer connects optional tools locally. Pin pilot versions, record environment and relevant inputs, and retain a rollback path. Avoid adding another orchestration framework unless an observed requirement needs it.

Compare baseline and candidate on the same held-out tasks with independent outcome grading. Record task success, critical/integrity failures, actual usage/cost coverage, active and waiting time, retries, and end-to-end latency including p95 when sample size permits. Missing measurements stay unknown. Inspect security findings and permission/data exposure before promotion. Reuse the ai-evaluation contract rather than counting retries as independent tasks.

The aspiration is complete accuracy and integrity for agreed tasks, with explicit speed budgets, scoped standards and security requirements. Finite trials cannot guarantee universal 100% accuracy or security. A passed test suite is evidence for those cases; standards compliance requires an identified standard, scope and assessment. No live-model accuracy, token reduction or production speed improvement has been established by this shortlist.